

Legal AI Assistant API — Integration Guide

This is the handoff guide for web, Android, and iOS clients. The live OpenAPI contract is always available at `GET /openapi.json`; Swagger UI is at `GET /docs`. Use those endpoints to generate a typed client when possible.

Files in this handoff

File	Purpose
<code>docs/API_INTEGRATION_GUIDE.md</code>	This guide — flow, auth, gotchas, examples
<code>docs/openapi.client.json</code>	OpenAPI 3 snapshot of the 112 client-facing routes (admin/internal routes removed). Import into Postman/Insomnia or generate a typed client (<code>openapi-generator</code> , <code>openapi-typescript</code> , etc.)

The snapshot was generated from the code on 2026-09-21. If the backend changes, re-export it from the running server (`GET /openapi.json`) and filter out `/admin*`, `/notarization/admin*`, and `/internal*`.

What you need from the backend owner before you start: the base URL for each environment (dev/staging/prod), and confirmation that your web origin is in the backend's `API_CORS_ORIGINS` (browser apps only — native Android/iOS requests are not subject to CORS).

Base URL and conventions

Set the base URL per environment, for example `https://api.example.com`. All routes below are rooted at that URL; this API currently has no `/v1` prefix. Send JSON requests with `Content-Type: application/json`. For authenticated requests add:

```
Authorization: Bearer <access_token>
```

`session_id` is the client-generated conversation identifier. Save the `session_id` returned by the first `/chat` response and send it on later turns. Do not send a filesystem path, database ID belonging to another user, or a user ID supplied by your own UI as a substitute for the JWT.

Errors raised by the application have this shape:

```
{ "error": { "code": "not_found", "message": "Message not found: 123", "details": {} } }
```

`code` is one of `bad_request`, `unauthorized`, `forbidden`, `not_found`, `draft_locked`, `draft_conflict`, `unsupported_export`, `rate_limited`, `internal_error`.

Not every error uses that shape. Parse defensively — these come back as FastAPI's default `{ "detail": ... }` instead:

Case	Status	Body
Malformed/missing request fields (FastAPI validation)	422	<code>{ "detail": [{ "loc": [...], "msg": "...", "type": "..." }] }</code>
Rate limit exceeded	429	<code>{ "detail": "Too many requests. Please try again shortly." }</code> plus a <code>Retry-After: 60</code> header
Voice routes (<code>/voice/*</code>): bad audio type / empty / too large / TTS down	415 / 400 / 413 / 502	<code>{ "detail": "..." }</code>

Write one error parser that reads `error.message`, falls back to `detail` (string, or the first `detail[] .msg`), and finally to a generic message. Status codes: 400 bad request, 401 missing/expired/revoked token, 403 access denied, 404 missing resource, 409 draft conflict, 413 file too large, 415 unsupported media type, 422 invalid fields, 429 rate-limited, 500 server error, 502 upstream (TTS) unavailable.

Limits and lifetimes

Item	Value
Access token lifetime	30 minutes
Refresh token lifetime	14 days (rotates on every use)
Rate limit	60 requests/minute per client IP (health routes exempt). Users behind one NAT/office IP share the budget — back off on 429
<code>question</code> length	1–8,000 characters
<code>/draft message</code> length	1–4,000 characters
Document upload	25 MB. Allowed: <code>.pdf .docx .txt .md .markdown .html .htm .json .csv .rtf .odt .png .jpg .jpeg .tiff .tif .bmp</code>
Voice audio	10 MB. Allowed types: <code>audio/wav, audio/mpeg, audio/webm, audio/ogg, audio/opus, audio/mp4, audio/x-m4a, audio/aac, audio/flac</code>

These values are configurable per environment; confirm them for production.

Which routes need a login

Access	Routes
Optional token (works anonymously in development; a token attaches the data to the account)	<code>/chat</code> , <code>/chat/stream</code> , <code>/history</code> , <code>/session*</code> , <code>/draft*</code> , <code>/draft-templates*</code> , <code>/documents*</code> , <code>/document-analysis</code> , <code>/feedback</code> , <code>/voice/*</code> , <code>/notarization/prepare</code> , <code>/notarization/requests</code> (create)
Token required in staging/production	<code>POST /upload</code> , <code>POST /documents/extract</code>
Token always required	<code>/cases/*</code> , <code>/assistant/preferences</code> , <code>/assistant/forms*</code> , <code>/assistant/jobs*</code> , <code>/assistant/downloads*</code> , <code>/assistant/search</code> , <code>DELETE /me/data</code> , <code>/notarization/signing/providers</code>
Public (no token)	<code>/health*</code> , <code>/register</code> , <code>/login</code> , <code>/refresh</code> , <code>/verify/{token}</code>

For a production app, always sign the user in first and send the bearer token on every call. An anonymous `session_id` is only reachable by whoever holds that id; a signed-in session is bound to the account.

Recommended app flow

- Call `GET /health/ready` before enabling the chat UI.
- Register or log in; securely store the access and refresh tokens.
- Send a user message to `POST /chat` or `POST /chat/stream`.
- Persist `session_id` and `message_id`; display `answer`, citations, warning, confidence, and workflow progress.
- For a private document, call `POST /upload` first, then ask the question in the same `session_id`, or use `POST /document-analysis` directly.
- Send user feedback using `POST /feedback`; offer session/data deletion in account settings.

Authentication

```
POST /register
{ "email": "user@example.com", "password": "at-least-10-characters", "full_name": "Asha Sharma" }
```

POST /login

```
{ "email": "user@example.com", "password": "at-least-10-characters" }
```

Both return:

```
{
  "access_token": "...",
  "refresh_token": "...",
  "token_type": "bearer",
  "role": "user",
  "user_id": "..."
}
```

POST /refresh

```
{ "refresh_token": "..." }
```

Refresh tokens rotate: replace both locally stored tokens with the returned values. Use `POST /logout` with the access-token header; pass `{ "refresh_token": "..." }` too when logging out on the current device.

Chat

POST /chat

```
{
  "question": "Mera cheque funds insufficient ke saath return hua hai. Ab kya karun?",
  "language": "hinglish",
  "session_id": "b7b83d75-9fef-4df4-ae9e-4b0a20be9dd2",
  "explanation_mode": "simple",
  "metadata_filters": {}
}
```

`question` is required (1–8,000 characters). `language` is optional; the API detects it when absent. `explanation_mode` is `simple`, `detailed`, or `advocate`. Do not set `user_id`; identity comes from the bearer token.

Important response fields:

```
{
  "message_id": "...",
  "session_id": "...",
  "answer": "...",
  "assistant_message": "...",
  "detected_language": "hinglish",
  "confidence": 0.84,
  "confidence_label": "High",
  "no_verified_context": false,
  "sources": [{
    "act_name": "...",
    "section": "...",
    "label": "...",
    "url": "...",
    "page_number": 12,
    "verification_status": "verified"
  }],
  "next_steps": [...],
  "risks": [...],
  "warnings": [],
  "disclaimer": "...",
  "draft": null,
  "artifact": null,
  "workflow_name": "",
  "progress_percentage": 0,
  "missing_field": null,
  "latency_ms": 0
}
```

UI rules:

- When `no_verified_context` is `true`, label the reply as unsupported; do not render it as a verified legal conclusion.
- Render each `sources[]` `label` and `url`; show page information when present.
- If `missing_field` exists, show one focused input instead of asking the user to repeat the whole matter.

• Use `draft`, `artifact`, `allowed_actions`, `requires_confirmation`, and `workflow_*` to drive drafting/document workflow UI.

- `retrieved_chunks` may contain source text. Treat it as optional debug/evidence content and do not expose it by default in a compact chat screen.

POST /chat/stream — Server-Sent Events

Use the same JSON body and authentication as `/chat`. Set `Accept: text/event-stream`. The server emits zero or more:

```
event: token
data: "partial response text"
```

The `data` of a `token` event is a **JSON-encoded string**, not an object — `JSON.parse(data)` gives the text. It may be a chunk of the answer or, for non-LLM replies (clarification questions, cached answers, refusals), the whole answer in one event. Append tokens to the bubble as they arrive.

The stream ends with exactly one event:

```
event: done
data: { "message_id": "...", "session_id": "...", "answer": "...", "sources": [...], ... }
```

The `done` payload is the full `ChatResponse` (same as `POST /chat`). **Replace the streamed text with `done.answer`** and take `sources`, `confidence`, `draft`, etc. from it — treat it as the authoritative result. If the connection drops before `done`, show a retry option and do not save the partial text as a finished answer.

Browser `EventSource` cannot send a POST body or an `Authorization` header, so on the web use `fetch()` with a streamed reader (or a library such as `@microsoft/fetch-event-source`). On Android use OkHttp SSE or a streaming Retrofit `ResponseBody`; on iOS use `URLSession.bytes(for:)`.

```
const res = await fetch(`${BASE}/chat/stream`, {
  method: "POST",
  headers: { "Content-Type": "application/json", Accept: "text/event-stream",
    Authorization: `Bearer ${accessToken}` },
  body: JSON.stringify({ question, session_id }),
});
const reader = res.body.getReader();
const decoder = new TextDecoder();
let buf = "";
for (;;) {
  const { value, done } = await reader.read();
  if (done) break;
  buf += decoder.decode(value, { stream: true });
  let i;
  while ((i = buf.indexOf("\n\n")) >= 0) {
    const frame = buf.slice(0, i); buf = buf.slice(i + 2);
    const event = /^event: (.*)$/m.exec(frame)?.[1];
    const data = JSON.parse(/^data: (.*)$/m.exec(frame)[1]);
    if (event === "token") appendToBubble(data); // data is a string
    else if (event === "done") finishBubble(data); // data is the ChatResponse
  }
}
```

If the same session already has a turn running, the server replies with a single `token + done` pair carrying a "turn in progress" message; disable the send button while a request is in flight.

Chat history and privacy

- `GET /history?session_id=<id>` — returns `{"session_id", "messages": [{"role", "content"}]}`.

Ordering gotcha: the list contains **all** user messages first, then **all** assistant messages (`user1...usern, assistant1...assistantn`), not interleaved. To render a transcript, split by `role` and zip the two halves by index. History holds question/answer text only — no sources or drafts.

- `GET /session?session_id=<id>` — current session state.
- `GET /session/facts?session_id=<id>` — confirmed facts, assumptions, missing details.
- `PATCH /session/facts?session_id=<id>` — body:

`{"kind": "confirmed", "key": "incident_date", "value": "2026-09-21"}`. `kind` is `confirmed` or `assumption`; `key` matches `[a-zA-Z0-9_.-]+`.

- `DELETE /session/facts/{kind}/{key}?session_id=<id>` — remove one fact.
- `PUT /session/missing-details?session_id=<id>` — body `{"details": ["..."]}`.
- `DELETE /chat?session_id=<id>` — deletes chat history and memory.
- `DELETE /session?session_id=<id>` — deletes all session data.
- `DELETE /me/data` — deletes the authenticated user's stored data.

Documents

POST /upload

Use `multipart/form-data`, not `JSON`.

```
file: <PDF/DOCX/image file>
session_id: <optional existing session id>
```

In staging/production, private uploads require a bearer token. Successful response:

```
{
  "document_id": "...",
  "filename": "notice.pdf",
  "status": "indexed",
  "chunks_indexed": 14,
  "detected_language": "english",
  "metadata": {},
  "warnings": []
}
```

After upload, include the same `session_id` in `/chat` and ask about the document. Other document routes:

- `GET /documents?session_id=<id>` — list accessible uploads.
- `DELETE /documents/{document_id}?session_id=<id>` — remove a private upload.
- `POST /document-analysis` — JSON body:

`{"document_id": "...", "analysis_type": "general", "language": "english", "session_id": "..."}.`

- `POST /documents/extract` — multipart file; transcribes/retypes without

publishing the document to the shared KB.

- `GET /documents/{document_id}/transcript/export?fmt=pdf|docx|txt&session_id=<id>`.

Never upload a user document to an admin KB route from the end-user app.

Drafting

For a conversational drafting experience, use `/chat`; its `draft` and `workflow` fields are the preferred public-client integration.

For a form-based drafting interface:

- `GET /draft-templates`, `GET /draft-categories`, `GET /draft-subcategories`

— populate template selection.

- `GET /draft-templates/search?q=<text>` — search templates.
- `POST /draft/recommend` — recommend a template from a user description.
- `POST /draft` — begin/continue a drafting turn. Body:

`{"session_id": "...", "message": "...", "language": "english"}; returns {"reply": "...", "draft": {...DraftTurnInfo}}` (same draft object as in `ChatResponse.draft`: `stage`, `missing_fields`, `sections`, `full_text`, `draft_id`, `available_export_formats`, `audit_findings`, `lifecycle_state`).

- `POST /draft/edit` — body `{"draft_id", "session_id", "instruction"}` or

`{"draft_id", "session_id", "target_field", "new_value"}`; optional `target_language` to translate.

- `POST /draft/audit` — body `{"draft_id", "session_id"}`; advisory findings on

text that goes beyond the facts the user supplied. Never blocks export.

- `POST /draft/validate` — validates **document* sections against a document

grammar*, not a stored draft: body `{"document_id", "sections": {...}, "context": {...}`, returns `{"status": "PASS|WARNING|ERROR", "issues": [...]}`.

- `POST /draft/approve`, `/draft/lock`, `/draft/unlock`, `/draft/rollback`,

`DELETE /draft/{draft_id}` — lifecycle. Edits to an approved/locked draft fail with `draft_locked` until it is unlocked.

- `POST /draft/export` — body

`{"draft_id","session_id","format":"pdf|docx|txt|rtf","watermark":false}`. This is a **POST that returns the file bytes** (not JSON). On mobile, save the response to a temp file and open the share sheet/viewer. PDF for some Indic scripts may return `unsupported_export` — offer DOCX/TXT instead.

- `GET /draft/{draft_id}/versions` — version history.

When `ChatResponse.artifact` is set it looks like `{"kind":"draft","label":"Download PDF","fmt":"pdf","download_path":"/draft/export","draft_id":"...", "media_type":"application/pdf"}`. `download_path` is an API route on the same base URL, never a filesystem path; call it with the bearer token (for `/draft/export`, as the POST above; notarized documents use `GET /notarization/documents/{document_id}/download`).

Read the precise request schemas from `/docs` before building direct form-based drafting, because template fields vary by document type.

Voice

- `POST /voice/chat` — `multipart/form-data: audio_file` (required),

`session_id`, `language`, `new_conversation` (bool). Speech-to-text → chat → text-to-speech in one call. Response:

```
`json { "session_id":"...", "transcribed_text":"...", "ai_response_text":"...", "audio_base64":"<base64 WAV or null>", "audio_format":"wav", "confidence":0.8, "intent":"...", "voice_final_confirmation_required":false, "draft":null } ``
```

`audio_base64` is null when text-to-speech is unavailable — always show `ai_response_text`. When a signed-in user omits `session_id`, the server reuses their most recent conversation; send `new_conversation=true` to start fresh. Voice responses do not include `sources` — use `/chat` when the UI needs citations.

- `POST /voice/speak` — `{"text":"..."}` → `{"audio_base64","audio_format":"wav"}`;

reads an existing answer aloud.

- `POST /voice/drafts/{draft_id}/confirm` — required before a voice-collected

draft is treated as confirmed. Body: `{"session_id":"...", "confirmation_text":"I CONFIRM THE REVIEWED FACTS"}` (the user must type/say exactly that phrase after reviewing the draft).

Signed-in user features (token always required)

Details and schemas for each are in `docs/openapi.client.json`.

- **Preferences** — `GET/PATCH/DELETE /assistant/preferences`.
- **Guided forms** — `POST /assistant/forms` (start), `GET /assistant/forms`,

`GET/PATCH /assistant/forms/{workflow_id}`, `POST /assistant/forms/{workflow_id}/confirm`.

- **Background jobs** — `POST/GET /assistant/jobs`, `GET /assistant/jobs/{job_id}`,

`POST /assistant/jobs/{job_id}/retry`. Poll a job every few seconds with backoff until its status is terminal.

- **Download center** — `GET /assistant/downloads` lists generated files;

`GET /assistant/downloads/{artifact_id}` downloads one (file bytes).

- **Search my content** — `GET /assistant/search?q=<2-200 chars>` searches the

user's own chats and uploaded documents.

- **Case management** — `POST/GET /cases`, `GET/PATCH/DELETE /cases/{case_id}`,

plus sub-resources: `/hearings`, `/notes`, `/documents`, `/drafts`, `/tasks`, `/timeline`, `/evidence/upload` (multipart), `/conflicts/resolve`, `/lawyer-summary`; `GET /cases/upcoming-hearings` and `GET /cases/hearing-reminders` for a dashboard/notification badge. Cases are private to the owning account.

Other end-user utilities

- `POST /summarize` — `{"text":"...", "max_chars":1200}` (text up to 120,000

characters; `max_chars` 200–5,000).

- `POST /search` — search the knowledge base without generating an answer.
- `POST /recommend-lawyer`, `POST /assistant/follow-up`,

`POST /assistant/legacy-reference` (map an old-code section, e.g. IPC/CrPC, to its current-law reference), `POST /workflows/cyber-fraud`, `/workflows/evidence/organize`, `/workflows/timeline`, `/workflows/jurisdiction` — task helpers; schemas in the OpenAPI file.

- **E-notarization** — `GET /notarization/eligibility`,

`POST /notarization/prepare`, `POST /notarization/requests`, `GET /notarization/requests`, `GET /notarization/documents/{id}/status`, `GET /notarization/documents/{id}/download`, and the public `GET /verify/{verification_token}` (target of the QR code on a notarized document). The notary-side routes (start-review, approve, reject, revoke, decision-token) belong to a notary console, not the citizen app. Ship this only after the backend owner confirms the feature is enabled for your environment.

Feedback, health, and operational routes

POST /feedback

```
{
  "session_id":"...",
  "message_id":"...",
  "rating":4,
  "category":"helpful",
  "comment":"Clear answer"
}
```

Allowed categories: `helpful`, `wrong_language`, `wrong_law`, `missing_source`, `unsafe_draft`.

- `GET /health/live` — process liveness.
- `GET /health/ready` — database and Redis readiness; use for deployment

health checks, not as a user-facing API.

- `GET /health` — diagnostic component status.

`/admin/*`, `/notarization/admin/*`, `/internal/metrics`, `/health/index-drift`, and KB maintenance routes are not end-user app routes and are excluded from `docs/openapi.client.json`. Keep them in a separate admin console, protect them with admin JWT roles, and never embed admin credentials in a browser or mobile app.

Client implementation notes

- Store access/refresh tokens in platform-secure storage. Do not put tokens

in URLs, analytics events, or application logs.

- On a 401, refresh once using `/refresh`, retry the original request once,

then send the user to sign-in if it still fails.

- Respect 429 responses with exponential backoff; do not retry validation or

permission errors automatically.

- Render server-provided text as plain text/Markdown with raw HTML disabled.
- Use HTTPS only in staging and production. Configure the deployed frontend

origin in `API_CORS_ORIGINS` on the backend.

- Every response carries an `x-request-id` header. Log it client-side and include it in bug reports so the backend team can trace the request. You may also send your own `x-request-id` header.
- A `/chat` turn can take several seconds (retrieval + LLM); the server's own ceiling per turn is 150 s (`chat_request_budget_seconds`, after which it returns a graceful timeout answer with `retryable: true`). Set your HTTP client timeout above that (e.g. 170 s), show a typing/progress state, and prefer `/chat/stream` for the main chat screen. If `retryable` is `true`, offer a "Try again" button.
- Do not claim the assistant has verified a legal answer when

`no_verified_context` is `true` or `sources` is empty.

Developer handover: project aur architecture

Purpose. Yeh FastAPI-based multilingual Indian legal assistant hai. Implemented capabilities: authenticated/anonymous chat, strict verified-KB RAG, Hindi/Hinglish language handling, private uploads, document analysis/extraction, conversational drafting and export, cases, feedback, voice, KB administration/automation, observability, aur optional e-notarization. `streamlit app/` developer/testing console hai; production consumer frontend **Not implemented / Not found in codebase**.

Layer	Actual implementation
Runtime	Python >=3.11,<3.13, FastAPI, Uvicorn; entry <code>app.main:app</code>
Persistence	MongoDB (<code>app/database/mongodb.py</code>) + filesystem under <code>storage/</code>
Cache/memory	Redis via <code>app/cache/redis_client.py</code> and <code>app/memory/store.py</code>
RAG	BGE-M3 embeddings, MongoDB Atlas/local cosine, BM25, RRF, legal reranking
LLM	<code>ollama</code> , <code>grog</code> , <code>openai</code> , <code>deepseek</code> , <code>openrouter</code> , <code>gemini</code> , <code>claude</code>
UI	Streamlit test console on 8501; API on 8000
Packaging	<code>pyproject.toml</code> , <code>requirements.txt</code> , Dockerfiles and Compose

```
User/client -> middleware (request ID, rate limit, CORS) -> API router
-> Auth/dependency -> ChatService / domain service
-> intent + language + entity extraction -> RAG retrieval
-> vector + BM25 -> RRF -> reranker -> verified context
-> resilient LLM provider -> answer/citations
-> Mongo chat/log repositories + Redis memory/cache -> response/SSE
```

`app/main.py` application lifecycle mein MongoDB/Redis connect karta hai, routers mount karta hai aur background KB workers/schedulers ko settings ke hisaab se start karta hai. Business orchestration `app/services/`, persistence boundaries `app/repositories/`, request/response contracts `app/schemas/`, aur collection names `app/models/collections.py` mein hain. `app/api/` mein route add karein; router ko `app/main.py` mein include karein. Generated/output/storage files ko source modules samajhkar edit na karein. Existing `.env` ko documentation ya VCS mein copy na karein; template `.env.example` hai.

Chat, session aur memory ka exact storage

Chat data is stored in MongoDB `chats` collection through `ChatHistoryRepository`; conversation working memory is stored in Redis and MongoDB **conversation memory**. User aur assistant turn separate `chats` documents hain; important fields include `session_id`, `user_id`, `role`, `content`, `created_at`. `sessions` lifecycle/session state rakhta hai. Client `session_id` de sakta hai; absent hone par service UUID banati hai. Authenticated ownership JWT `user_id` se enforce hoti hai, request body ke claimed user se nahi.

`MemoryStore` Redis mein session state/history cache karta hai; durable facts and conversation summaries `conversation_memory` repository se MongoDB mein rehte hain. Redis unavailable ho to code repository/in-process degradation paths use karta hai, lekin `/health/ready` Redis ko readiness dependency maanta hai. `DELETE /chat/history` + memory clear karta hai; `DELETE /session` wider session data clear karta hai; `DELETE /me/data` authenticated account data-erasure flow hai. `/chat/stream` SSE transport hai: same final `ChatResponse` persist hota hai, sirf delivery token events + final `done` event mein hoti hai.

RAG aur legal Knowledge Base

```
PDF/DOCX/text/image -> DocumentLoader -> OCR when required -> clean text
-> LegalTextChunker -> metadata/quality gates -> BGE-M3 embedding (1024 dims)
-> MongoDB embeddings metadata + BM25 index
-> query rewrite/multilingual bridge -> Atlas $vectorSearch (local fallback)
-> BM25 -> reciprocal_rank_fusion(k=60) -> LegalReranker
-> approved/verified evidence -> prompt -> LLM -> page/source citations
```

Defaults `app/core/config.py` se: BAAI/bge-m3, dimension 1024, batch 32, Atlas index `legal_chunks_vector_index`, local scan cap 20000, optional neural reranker `cross-encoder/ms-marco-MiniLM-L-6-v2` (default off, weight 0.25). Chunk size/overlap hardcoded single global number nahi hai: `app/rag/chunker.py` legal structure-aware boundaries aur per-document behavior use karta hai; runtime claim ke liye isi module ko authority maanein. Metadata includes document/source identity, act/section, language, jurisdiction/applicability, page evidence, verification/review status and ownership fields.

Only approved/verified shared KB context confident legal answer path mein jaata hai. Empty verified retrieval par unverified excerpt optional disclosure ho sakta hai (`chat_unverified_disclosure_enabled=true`). Controlled GK answer default off hai (`general_knowledge_fallback_enabled=false`), specific/current provision ke liye allowed nahi, citation-shaped output reject hota hai. Missing-Act auto-fetch opt-in hai (`kb_gap_autofetch_enabled=false`). Citation assembly `app/rag/citation.py`; relevance/confidence guards `relevance.py`, `answer_quality.py`, `confidence.py`; no evidence ko fabricated section/page se fill nahi kiya jaata.

Document aur Act ingestion

Private `POST /upload` -> malware/extension/size checks -> `storage/uploads` -> parse/chunk/embed -> owner-scoped Mongo records. Admin shared KB upload alag `KnowledgeBaseIngestionService` path hai: stage -> SHA-256 claim -> exact/near duplicate detection -> quality/provenance review -> index -> transfer to `storage/knowledge_base`; duplicates `storage/archive`, failures/unverified items `storage/kb_review`, transient bytes `storage/kb_staging`. Private uploads automatic shared-KB promotion nahi hote.

Code mein **31 concrete catalogue adapters** mile: 11 central (`IndiaCodeAdapter`, `LegislativeDepartmentAdapter`, `EGazetteAdapter`, `SupremeCourtAdapter`, `RBIAdapter`, `SEBIAdapter`, `MCAAdapter`, `IRDAIAdapter`, `CBDTAdapter`, `CBICGSTAdapter`, `SelectedMinistriesAdapter`), 18 State/UT adapters in `kb_state_adapters.py`, aur 2 High Court adapters (Delhi, Bombay). Manifest `config/kb_sources.json`; sync/worker entry points `scripts/sync_official_kb_sources.py`, `scripts/run_phase3_worker.py`. Automation download, domain/TLS, MIME/size, malware and provenance gates ke baad `needs_review/machine` verification path use karti hai. OCR Tesseract `eng+hin`; low-confidence scan par configured Gemini vision fallback hai.

Database, indexes aur filesystem map

Mongo database default `legal_ai_assistant`. Canonical names `app/models/collections.py` mein hain:

Data	Collection(s)
Users/auth	<code>users</code> ; refresh/session state in <code>sessions</code>
Chat/memory	<code>chats</code> , <code>conversation_memory</code>
Documents/RAG	<code>uploaded_documents</code> , <code>embeddings_metadata</code> , <code>document_versions</code> , <code>document_transcripts</code>
Drafting/cases	<code>legal_drafts</code> , <code>draft_versions</code> , <code>cases</code>
KB operations	<code>kb_staging_records</code> , <code>kb_automation_jobs</code> , <code>kb_automation_state</code> , <code>kb_gap_autofetch_queue</code> , <code>kb_source_adapters</code> , <code>legal_sources</code> , <code>indexing_jobs</code>
Feedback/telemetry	<code>feedback</code> , <code>prompt_logs</code> , <code>query_logs</code> , <code>system_logs</code> , <code>observability_events</code> , <code>intent_events</code> , <code>intent_feedback</code> , <code>audit_logs</code> , <code>operational_events</code>
User workflows	<code>user_preferences</code> , <code>form_workflows</code> , <code>background_jobs</code> , <code>download_artifacts</code> , <code>evaluation_runs</code>
Notarization	<code>notarization_documents</code> , <code>notary_accounts</code> , <code>notarization_requests</code> , <code>esign_sessions</code> , <code>notarization_audit_events</code>

`scripts/create_indexes.py` is idempotent index authority: unique user email, chat session timeline/user history, document hash/source filters, act/section and jurisdiction filters, draft/case ownership, KB queues, audit timelines, plus TTL indexes for `sessions`, `conversation_memory`, and configured log retention. Atlas vector index definition bhi isi script mein hai. Relationships Mongo foreign keys nahi: string/ObjectId references such as `user_id`, `session_id`, `document_id`, `draft_id`, `case_id` application repositories enforce karte hain.

Filesystem path	Meaning
storage/uploads	private user originals
storage/knowledge_base	accepted shared KB originals
storage/kb_staging / storage/kb_review	transient ingestion / quarantine-review
storage/archive	recoverable duplicate originals
storage/drafts	generated draft exports
storage/operations / storage/backups	reconciliation artifacts / Mongo backups

Cloud object storage adapter **Not implemented / Not found in codebase**. Cleanup script `scripts/document_storage_cleanup.py`; Mongo backup/restore scripts exist.

LLM, drafting aur security

LLMFactory provider string ko seven implementations mein map karta hai; `create_resilient()` primary + comma-separated `LLM_FALLBACK_PROVIDERS`, retries and backoff apply karta hai. Chat defaults: max 4096 tokens, temperature 0.2, `top_p=0.9`, `top_k=40`, provider timeout 120 s, total chat budget 150 s. Streaming provider support transport/provider dependent hai; public SSE contract `/chat/stream` stable boundary hai. New provider ke liye `app/llm/base.py` contract implement, module add, `factory.py` mapping/settings add, tests add karein.

Drafting mein 55 YAML templates `app/drafting/templates/` mein hain. Flow: intent/discovery -> template loader -> `DraftConversationManager` field collection -> validation/safety/fact audit -> LLM generation -> `legal_drafts` + version in `draft_versions` -> lifecycle approve/lock/rollback -> exporter. Export files `storage/drafts`; PDF/DOCX/TXT/RTF route bytes return karta hai. Hindi/Indic PDF font config `PDF_UNICODE_FONT_PATH/PDF_SCRIPT_FONT_PATHS`; unsupported reliable render case mein API alternate format signal kar sakti hai.

Passwords security helpers se hash hote hain; JWT HS256 default, access 30 min, refresh 14 days with rotation/revocation. Roles dependencies admin/notary routes protect karte hain. Middleware request ID, trusted-proxy-aware IP rate limiting (default 60/min), structured errors and CORS apply karta hai. Upload size/type, filename/path isolation, malware scan, prompt-injection and draft-safety checks implemented hain. Production startup weak `JWT_SECRET_KEY` reject karta hai. Reverse proxy/TLS component compose mein **Not implemented / Not found in codebase**; production mein external HTTPS proxy/LB required hai.

Setup, deployment, logging aur testing

```
py -3.11 -m venv .venv
.venv\Scripts\pip.exe install -e ".[dev]"
Copy-Item .env.example .env
docker compose up -d mongo redis
.venv\Scripts\python.exe scripts/create_indexes.py
.venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000
.venv\Scripts\python.exe -m streamlit run streamlit_app/app.py --server.port 8501
```

Alternative complete stack: `docker compose up --build`; production definition `docker-compose.production.yml`. Health: `/health/live`, `/health/ready`, `/health`. Structured logging `app/core/logger.py`; request correlation `x-request-id`; optional `SENTRY_DSN` and `OTEL_EXPORTER_OTLP_ENDPOINT`. Application log file sink **Not implemented / Not found in codebase**; process/ container stdout-stderr collect karein.

Environment minimum: `MONGODB_URI`, `MONGODB_DATABASE`, `REDIS_URL`, strong `JWT_SECRET_KEY`, `API_CORS_ORIGINS`, chosen `LLM_PROVIDER` and corresponding key/base/model variables. RAG knobs: `EMBEDDING_MODEL`, `EMBEDDING_DIMENSIONS`, `VECTOR_SEARCH_BACKEND`, `MONGODB_VECTOR_INDEX`. `Storage/OCR/automation/voice/` drafting variables exact names `.env.example` and `Settings` fields (case-insensitive environment mapping) mein documented hain; real secrets kabhi client bundle mein na dein.

Testing pytest-based hai; `tests/` mein 159 `test_*.py` modules plus benchmark fixtures hain. Run `.venv\Scripts\python.exe -m pytest -q`; focused suites by file/`-k`; `scripts/run_tests.ps1` wrapper available. `conftest.py` external network ko loopback ke alawa block karta hai. Repository ka last reported run documentation banate waqt execute nahi kiya gaya, isliye current pass count **Not verified in this document**.

Problem	Likely cause	Check/fix
<code>/health/ready</code> fails	Mongo/Redis down	URLs check; Compose services start
401/403	expired/revoked JWT or role	refresh once; token role/ownership inspect
Empty/unsupported answer	no approved KB evidence	source review/index status and filters check
Slow RAG	Atlas fallback/local scan or LLM	vector index, metrics, provider timeout check
OCR weak	missing Tesseract language/Gemini key	<code>OCR_LANGUAGE</code> , confidence, fallback config
PDF glyph issue	missing script font/export limit	font paths; DOCX/TXT fallback use
Act sync fails	TLS/domain/MIME/malware gate	automation job/review reason inspect
429	per-IP rate limit	<code>Retry-After</code> ; trusted proxy config verify

Feature-change map aur final system map

Change	Primary files
API/schema	<code>app/api/</code> , <code>app/schemas/</code> , <code>app/main.py</code>
Act/state adapter	<code>app/services/kb_*</code> <code>adapters.py</code> , registry/manifest, adapter tests
LLM	<code>app/llm/</code> , <code>app/core/config.py</code>
Embedding/chunk/retrieval	<code>app/rag/embeddings.py</code> , <code>chunker.py</code> , <code>vector_store.py</code> , <code>retriever.py</code> , <code>reranker.py</code>
Draft type	new YAML in <code>app/drafting/templates/</code> ; loader auto-discovers
Collection/index	<code>app/models/collections.py</code> , <code>repository</code> , <code>scripts/create_indexes.py</code>
Language	<code>app/language/</code> , multilingual RAG/prompts, translation maps/tests

```
AUTH: register/login -> users -> JWT/refresh -> bearer dependency
INGEST: official/private bytes -> validate/OCR -> chunk/embed -> Mongo/BM25 -> review
AUTO-ACT: adapter/manifest -> fetch/security gates -> staging -> verify -> index
CHAT: user -> auth -> intent/entities -> RAG -> rerank -> LLM -> citations -> chats/memory
DRAFT: chat intent -> YAML template -> collect facts -> generate/audit -> versions -> approve
PDF: stored draft -> exporter + script font -> integrity guard -> byte response
```

API ka exhaustive machine-readable contract, request/response schemas, examples aur status codes `docs/openapi.client.json` mein hain; runtime source of truth `GET /openapi.json` hai. Admin/internal routes intentionally client snapshot se excluded hain aur `app/api/admin*.py` mein review karne chahiye.